*****What are the differences between connection-less and connection oriented communications?**

Layers can offer two different types of services to the layers above them.

Connection oriented service is one in which the user first establishes a connection, uses the connection and the releases the connection. The essential aspect of a connection is that it acts like a tube, the sender pushes the objects(bits) in at one end, the receiver takes them out at the other end. In most cases, the order is preserved so that the bits arrive in the order they were sent. This service is analogous to telephone system. In this case, the user and subnet conduct a negotiation about parameters to be used such as maximum message size, quality of service.

In connection-less service, each message carries the full destination address and each one is routed though the system independent of all the others. In case of message sending and receiving the order is usually maintained, though, exception is common.

***Distinguish between TCP and UDP.

Transfer Control Protocol(TCP)	User Datagram Protocol(UDP)
This protocol is connection oriented protocol.	This protocol is connection-less protocol.
It is reliable.	It is unreliable.
Ensures accurate, successive delivery of packets.	Does not ensure accurate, successive delivery of packets.
More secured protocol.	Less secured protocol.

***Contrast between *Congestion Control* and *Flow Control*.**

Congestion occurs when the receiving machine gets data more than his receiving capacity. This problem leads to mechanisms for disassembling, transmitting and reassembling message.

On the other hand, *Flow Control* is an issue that occurs at every level considering how to keep a fast sender from swamping a slow receiver with data.

Thus, *Congestion Control deals with the capacity of data* while *Flow Control takes into account the rate of data transfer*.

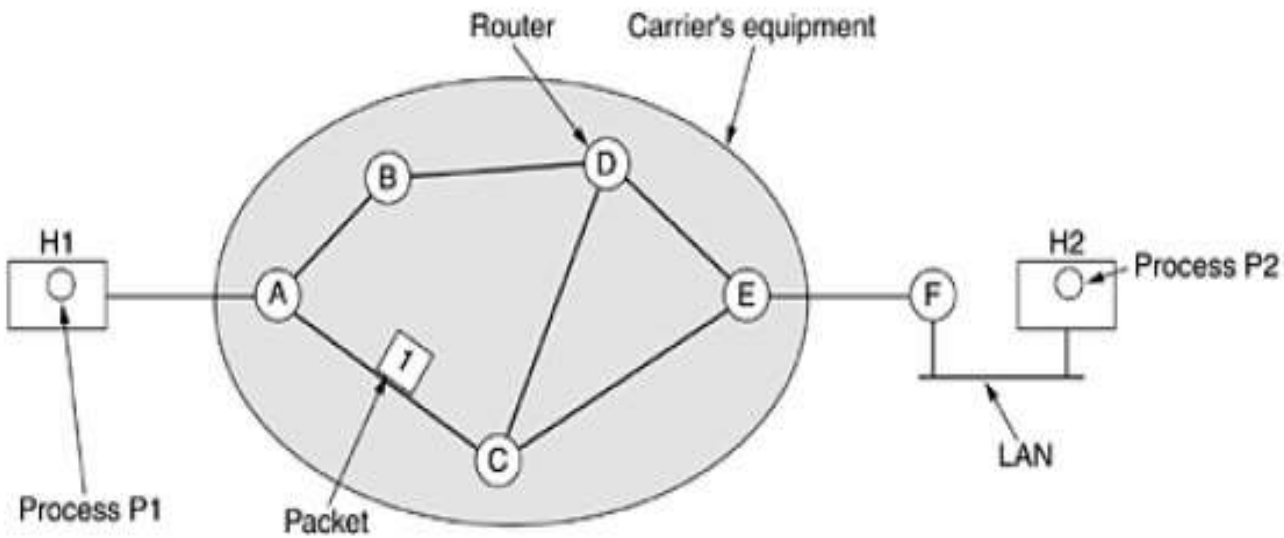
Differentiate between packet switching and circuit switching.

In packet switching, there is no dedicated link between two hosts. This mode has the probability of occurrence of congestion. The example of packet switching is *internet*.

On the contrary, circuit switching mode requires a dedicated link between two terminals as done in telephone network. There cannot occur congestion.

What do you mean by *store-and-forward packet switching*?

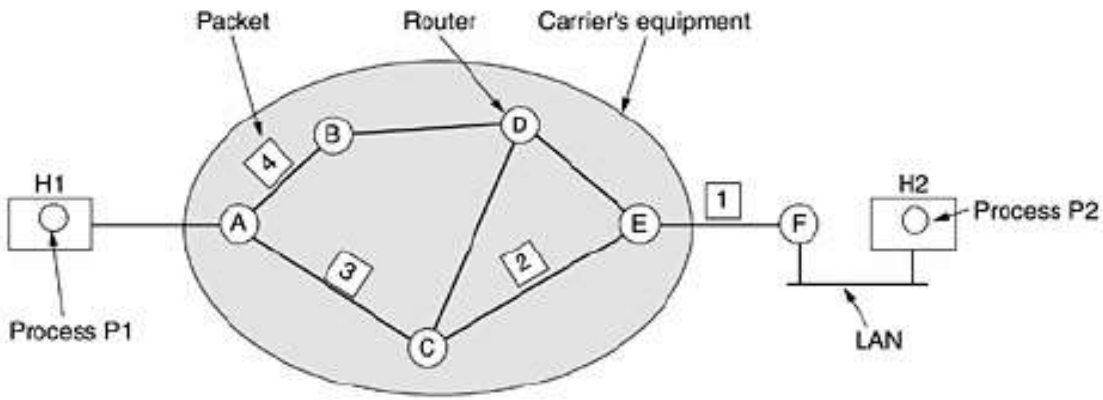
Let us consider the following phenomenon where a host H1 is directly connected to a carrier router and another host H2 is connected to a local router via a LAN.



Now, let a host has a packet to send through the network to another host either on its own LAN or a point-to-point link to the carrier. The packet is stored there until it has fully arrived so the checksum can be verified. Then it is forwarded to the next router along the path until it reaches the destination host, where it is delivered. This mechanism is known as *store-and-forward packet switching*.

Explain *Virtual Circuit* and *Datagram* Subnet briefly. Write down the difference between *Virtual Circuit* and *Datagram* Subnet.

There are two classes of services the network layer can provide to its users. If connectionless service is offered, packets are injected into the subnet individually and routed independently of each other. No advance setup is needed. In this context, the packets are frequently called *datagrams* and the subnet is called a *datagram subnet*.



A's table

initially		later		C's table		E's table	
A	–	A	–	A	A	A	C
B	B	B	B	B	A	B	D
C	C	C	C	C	–	C	C
D	B	D	B	D	D	D	D
E	C	E	B	E	E	E	–
F	C	F	B	F	E	F	F

Dest. Line

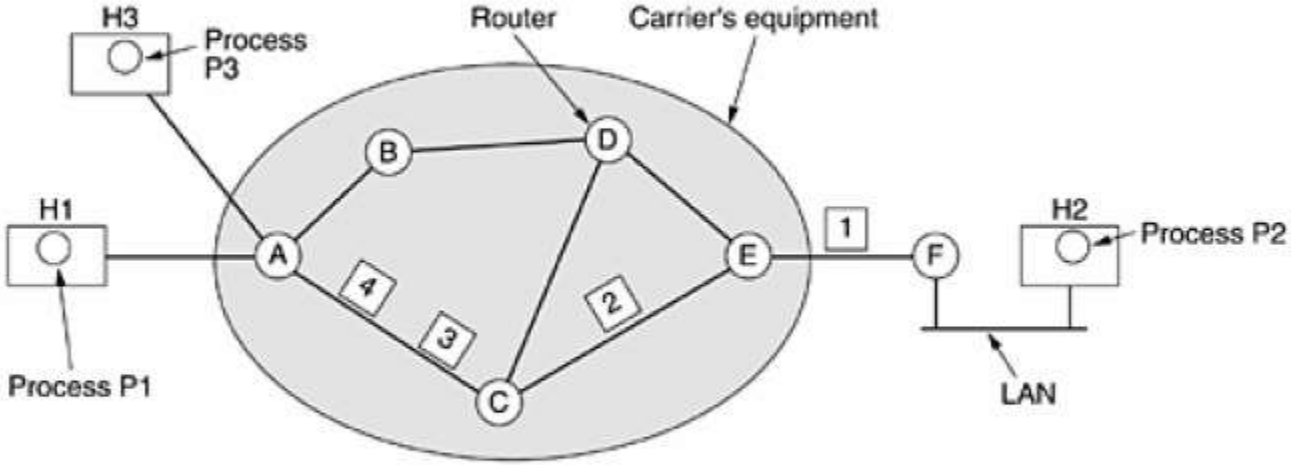
Difference between *Virtual Circuit* and *Datagram* Subnet.

Issue	Datagram subnet	Virtual-circuit subnet
Circuit setup	Not needed	Required
Addressing	Each packet contains the full source and destination address	Each packet contains a short VC number
State information	Routers do not hold state information about connections	Each VC requires router table space per connection
Routing	Each packet is routed independently	Route chosen when VC is set up; all packets follow it
Effect of router failures	None, except for packets lost during the crash	All VCs that passed through the failed router are terminated
Quality of service	Difficult	Easy if enough resources can be allocated in advance for each VC
Congestion control	Difficult	Easy if enough resources can be allocated in advance for each VC

What is the difference between *routing* and *forwarding*?

Each router has a table containing information about the other routers in the subnet. We can say that a router has two processes inside it. One process is responsible for receiving packets, looking for the appropriate outgoing path in the routing table and then sending the packet through the correct outgoing line. This is called *forwarding*. The other process, on the other hand, updates the contents of the routing table considering the traffic in the network and other issues. This process is called *routing*.

If connection-oriented service is used, a path from the source router to the destination router must be established before any data packets can be sent. This connection is called a *virtual circuit* and the subnet is called a *virtual-circuit subnet*.



A's table

In		Out	
H1	1	C	1
H3	1	C	2

C's table

A	1	E	1
A	2	E	2

E's table

C	1	F	1
C	2	F	2

Define and classify routing algorithm. Differentiate between *dynamic routing* and *static routing*.

The routing algorithm is the software part of the network layer that decides on which output line an incoming packet should be transmitted.

The routing algorithms can be grouped into two major classes.

(i) Non-adaptive Routing (Static Routing)

(ii) Adaptive Routing (Dynamic Routing)

Difference between dynamic and static routing:

Static routing is a predefined form of routing that does not change the route for a packet depending on the current network traffic and topology. That is, it never looks which path of the network is busy/free, it just chooses the path as defined earlier.

On the other hand, dynamic routing changes the route for a particular packet depending on the current network traffic and topology. That is, it takes into account which path of the network is currently busy/free and chooses path for a packet accordingly.

State the *Optimality Principle.*

We can make a general statement about optimal routes without regard to network topology or traffic. This statement is known as the *optimality principle*. It states that if router J is on the optimal path from router I to router K, then the optimal path from J to K also falls along the same route.

For example, let us consider the part of the route from I to J is r_1 and the rest of the route is r_2. If a route better than r_2 existed from J to K, it could be concatenated with r_1 to improve the route from I to K, contradicting our statement that r_1r_2 is optimal.

Define *Sink Tree.*

The set of optimal routes from all sources to a given destination form a tree rooted at the destination. Such a tree is called a *sink tree* as illustrated in the following figure. A sink tree is not necessarily unique; other trees with the same path lengths may exist. The goal of all routing algorithms is to discover and use the sink trees for all routers.

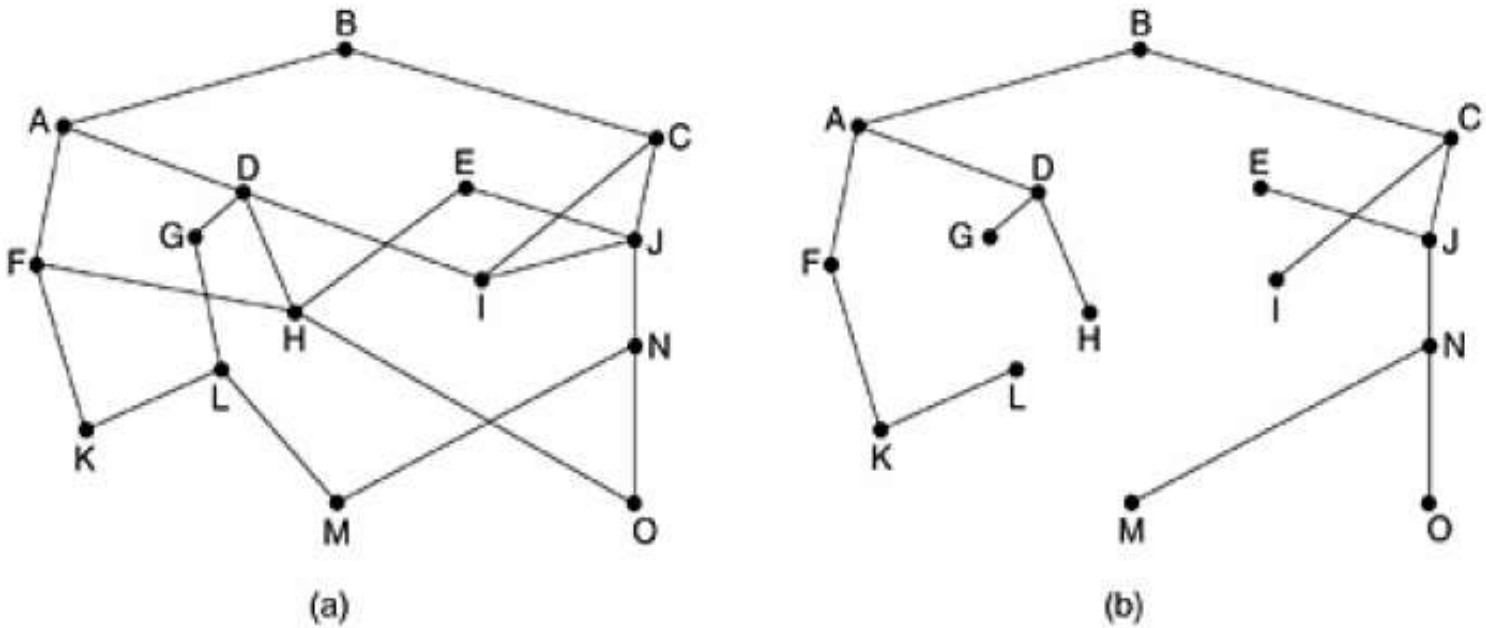
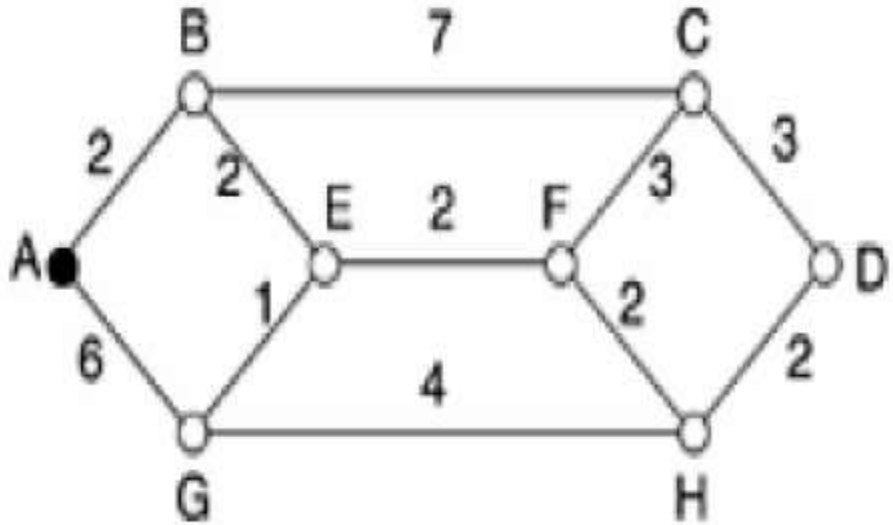


Fig.: (a) a subnet (b) sink tree for router B

Describe shortest path. Using the Dijkstra's algorithm find the shortest path from A to D of the following network.



Shortest Path Routing:

In case of routing, we should have some initial consideration.

Graph → Subnet

Node → Router

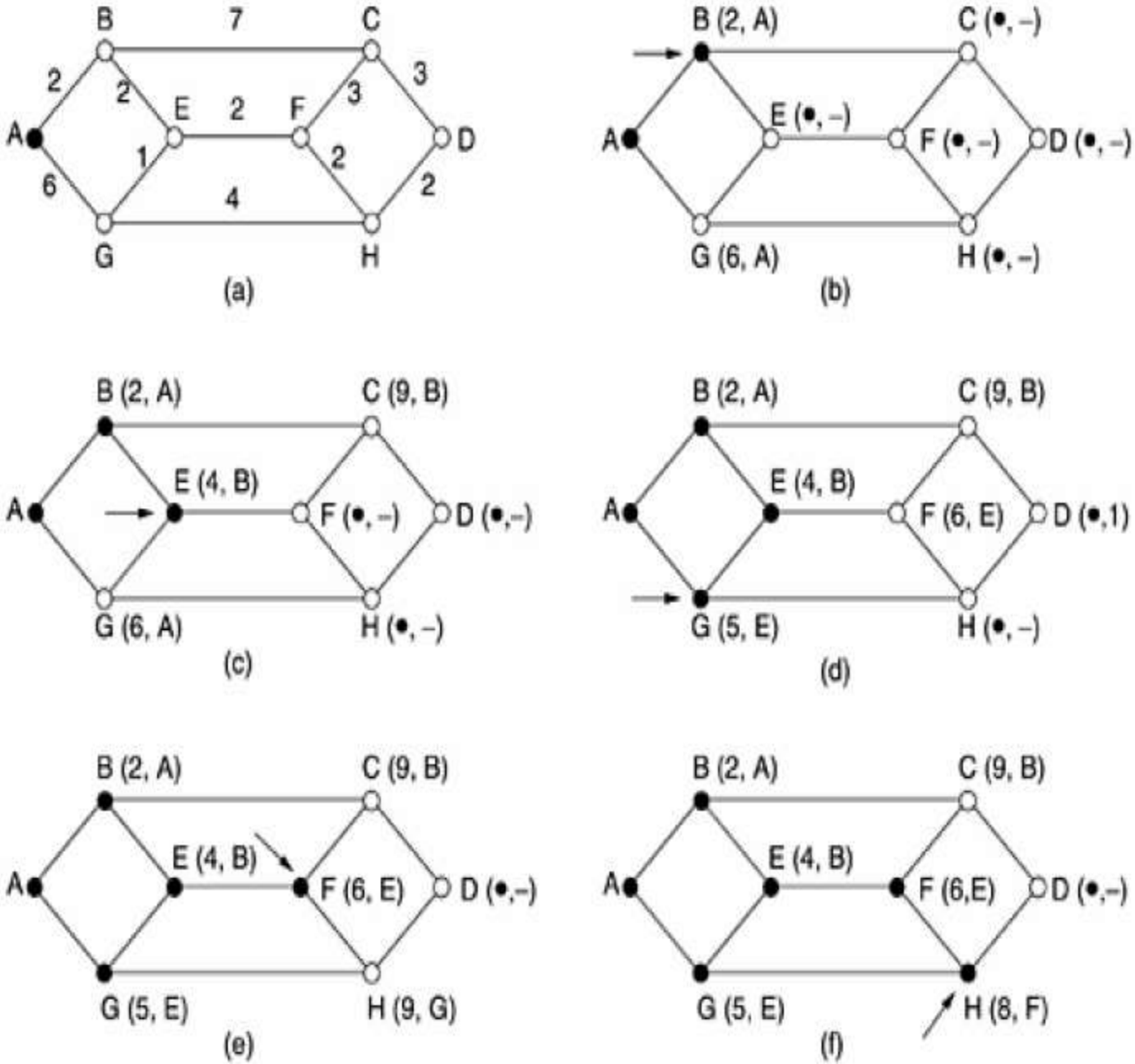
Arc → Communication Link or Link

Label → Geographic Distance, Bandwidth, Average Traffic, Communication Cost, Mean Queue Length, Measured Delay, and other factors.

To choose a route between a given pair of routers, the shortest path algorithm just finds the shortest path between them on the graph.

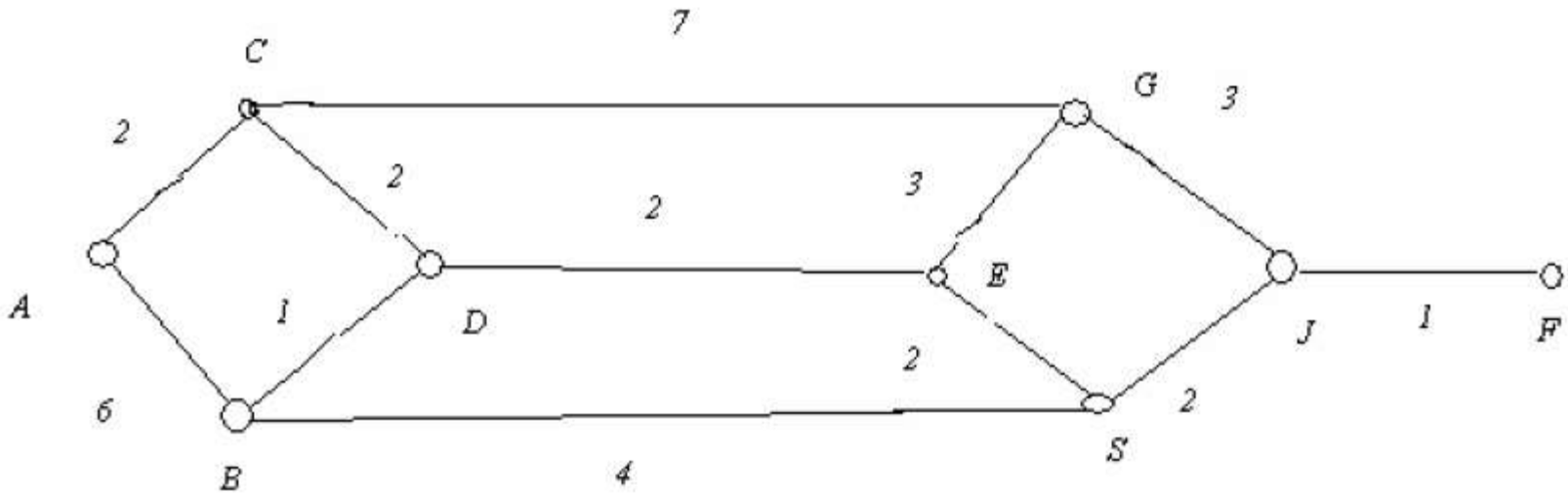
Shortest Path Finding using Dijkstra's algorithm:

The following figure shows how the algorithm is applied on the given problem. Each node *n* is labeled with the shortest distance from the source node and the node from which the *n* is reached. Initially all distances are unknown and thus marked as *infinity*. A labeled distance can be either tentative or permanent. A distance is marked permanent as soon as it is discovered as the shortest value.



[NOTE: Shortest Path Routing is an example of *non-adaptive (static)* routing]

Using the Dijkstra's algorithm find the shortest path from A to F of the following network.

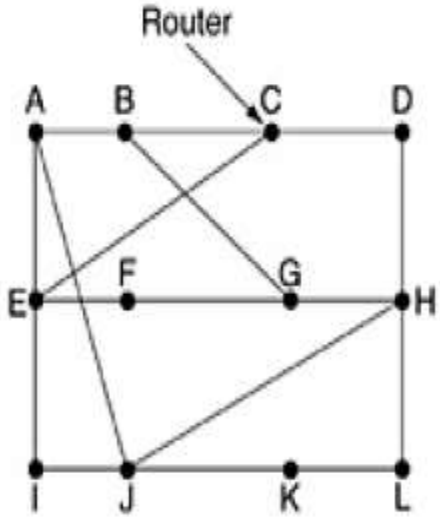


With an appropriate example explain the distance vector routing algorithm.

In distance vector routing, each router maintains a routing table containing one entry for each router in the subnet. This entry contains two parts: the preferred outgoing line to use for that destination and an estimate of the time or distance to that destination. These tables are updated by exchanging information with the neighbors. The metric used

might be number of hops, time delay in milliseconds or total number of packets queued along the path.

Let us assume that delay is used as a metric and that the router knows the delay to each of its neighbors. Once every T msec each router sends to each neighbor a list of its estimated delays to each destination. It also receives a similar list from each neighbor. Imagine that one of these tables has just come in from neighbor X, with X_i being X's estimate of how long it takes to get to router i. If the router knows that the delay to X is *m* msec, it also knows that it can reach router i via X in X_i+m msec. By performing this calculation for each neighbor, a router can find out which estimate seems the best and use that estimate and the corresponding line in its new routing table. This updating process for a router J is illustrated in the following figure.



(a)

To	A	I	H	K	New estimated delay from J	Line
A	0	24	20	21	8	A
B	12	36	31	28	20	A
C	25	18	19	36	28	I
D	40	27	8	24	20	H
E	14	7	30	22	17	I
F	23	20	19	40	30	I
G	18	31	6	31	18	H
H	17	20	0	19	12	H
I	21	0	14	22	10	I
J	9	11	7	10	0	–
K	24	22	22	0	6	K
L	29	33	9	9	15	K
	JA delay is 8	JI delay is 10	JH delay is 12	JK delay is 6		

Vectors received from J's four neighbors

New routing table for J

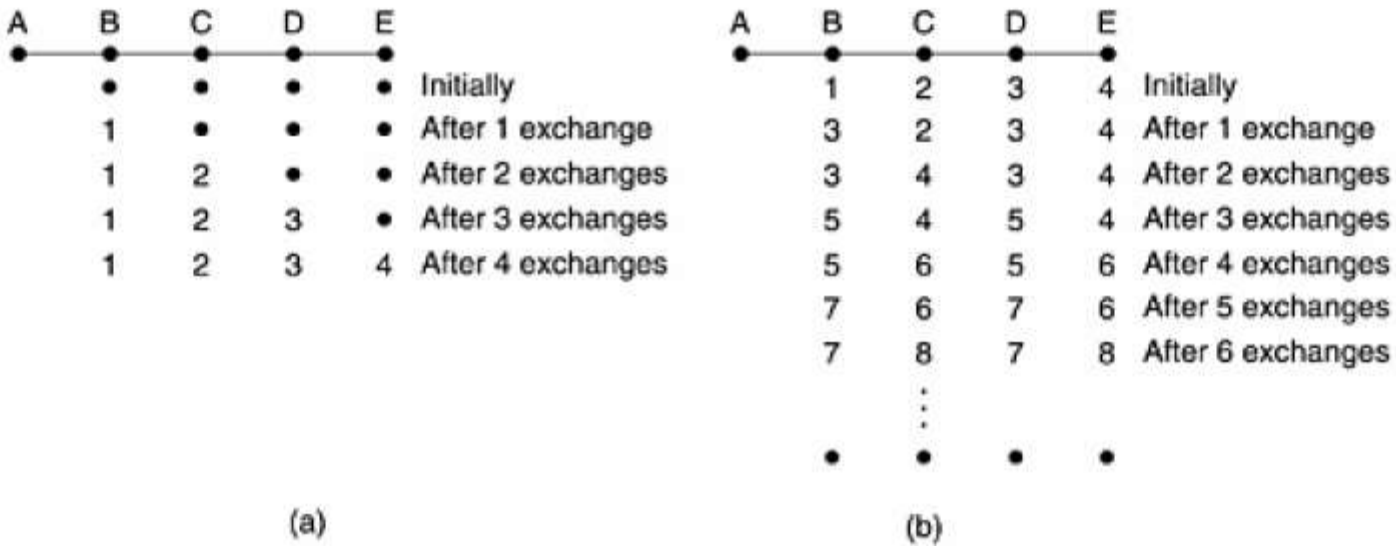
(b)

Part (a) shows a subnet. The first four columns of part (b) show the delay vectors received from the neighbors of router J. A claims to have a 12-msec delay to B, a 25-msec delay to C, a 40-msec delay to D, etc. Suppose that J has measured or estimated its delay to its neighbors, A, I, H, and K as 8, 10, 12, and 6 msec, respectively. Consider how J computes its new route to router G. It knows that it can get to A in 8 msec, and A claims to be able to get to G in 18 msec, so J knows it can count on a delay of 26 msec to G if it forwards packets bound for G to A. Similarly, it computes the delay to G via I, H, and K as 41 (31 + 10), 18 (6 + 12), and 37 (31 + 6) msec, respectively. The best of these values is 18, so it makes an entry in its routing table that the delay to G is 18 msec and that the route to use is via H. The same calculation is performed for all the other destinations, with the new routing table shown in the last column of the figure.

[NOTE: Distance Vector Routing is an example of *adaptive (dynamic)* routing]

Explain the count-to-infinity problem of the distance vector routing algorithm.

In case of distance vector routing, the algorithm works well for *good news*, but responds slow for *bad news*. In latter case, the propagation is too slow to decide any path practically, thus it leads to a problem of infinite time which is known as *count-to-infinity problem*. The situation is explained by the following figures.



Let, in our example, the delay is number of hops. For good news propagation, we consider figure (a). Suppose A is down initially and all the other routers know this. When A comes up, the other routers learn about it via the vector exchanges. At the time of the first exchange, B learns that its left neighbor has zero delay to A. B now makes an entry in its routing table that A is one hop away to the left. All the other routers still think that A is down. At this point, the routing table entries for A are as shown in the second row of the figure. On the next exchange, C learns that B has a path of length 1 to A, so it updates its routing table to indicate a path of length 2, but D and E do not hear the good news until later. Clearly, the good news is spreading at the rate of one hop per exchange. In a subnet whose longest path is of length N hops, within N exchanges everyone will know about newly-revived lines and routers.

But for bad news propagation like in figure (b), let, all the lines and routers are initially up. Routers B, C, D, and E have distances to A of 1, 2, 3, and 4, respectively. Suddenly A goes down. At the first packet exchange, B does not hear anything from A. Fortunately, C says: Do not worry; I have a path to A of length 2. Little does B know that C's path runs through B itself. For all B knows, C might have ten lines all with separate paths to A of length 2. As a result, B thinks it can reach A via C, with a path length of 3. D and E do not update their entries for A on the first exchange. On the second exchange, C notices that each of its neighbors claims to have a path to A of length 3. It picks one of them at random and makes its new distance to A 4, as shown in the third row of figure. Thus, there is no router having a value more than one higher than the minimum of all its neighbors.

If the metric used is numeric, then this infinite level can be prevented by making *infinity = longest path+1*, but if it is time delay, it is almost impossible to find a solution since it has no upper-bound. However, one solution can be possible- since C first learns about A from B, it cannot update any information about A to B.